1. Aim of the practical: To develop a secure User Authentication System using Node.js, Express.js, and MongoDB that allows users to register, log in, and access protected routes using JWT (JSON Web Tokens) for session management.

2. Objective:

- To implement user registration and login functionality in a Node.js application.
- To integrate MongoDB for storing user data securely.
- To use password hashing for security using bcrypt.js.
- To implement JWT-based authentication for protected routes.
- To understand middleware implementation for authorization.

3. Algorithm/Flowchart:

1. Initialize Project
    - Create a new Node.js project using npm init.
    - Install dependencies: express, mongoose, dotenv, bcryptjs, jsonwebtoken, and nodemon.
2. Setup Server
    - Create app.js and configure Express, dotenv, and MongoDB connection using Mongoose.
3. Create Model
    - Define a User schema in /models/Students.js containing name, email, and password.
4. Create Controller
    - In /controllers/authControllers.js, implement:
        - registerUser → to hash password and save new user.
        - loginUser → to verify credentials and generate JWT.
5. Setup Routes
    - In /routes/authRoutes.js, define routes for /register and /login mapped to controller functions.
6. Implement Middleware
    - In /middleware/auth.js, verify the JWT token to protect routes.

7. Run and Test
    - Start server using npm run dev.
    - Test endpoints using Postman:
        - POST /api/auth/register
        - POST /api/auth/login
        - GET /api/protected (with JWT token)
8. Verify Output
    - Check MongoDB to confirm registered users and verify authentication flow works correctly.

App.js

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');

const authRoutes = require('./routes/authRoutes');
const studentRoutes = require('./routes/studentRoutes');

const app = express();
app.use(express.json());

// Routes
app.use('/api/auth', authRoutes);
app.use('/api/students', studentRoutes);

// basic root
app.get('/', (req, res) => res.send('University Course App - Authentications'));

// connect to MongoDB and start server
const PORT = process.env.PORT || 5000;
const MONGO_URI = process.env.MONGO_URI || 'mongodb://127.0.0.1:27017/university';

mongoose.connect(MONGO_URI, { useNewUrlParser: true, useUnifiedTopology: true })
  .then(() => {
    console.log('MongoDB connected');
    app.listen(PORT, () => console.log(`Server running on port ${PORT}`));
  })
  .catch(err => {
    console.error('Mongo connection error:', err.message);
    process.exit(1);
  });
```

Auth.js

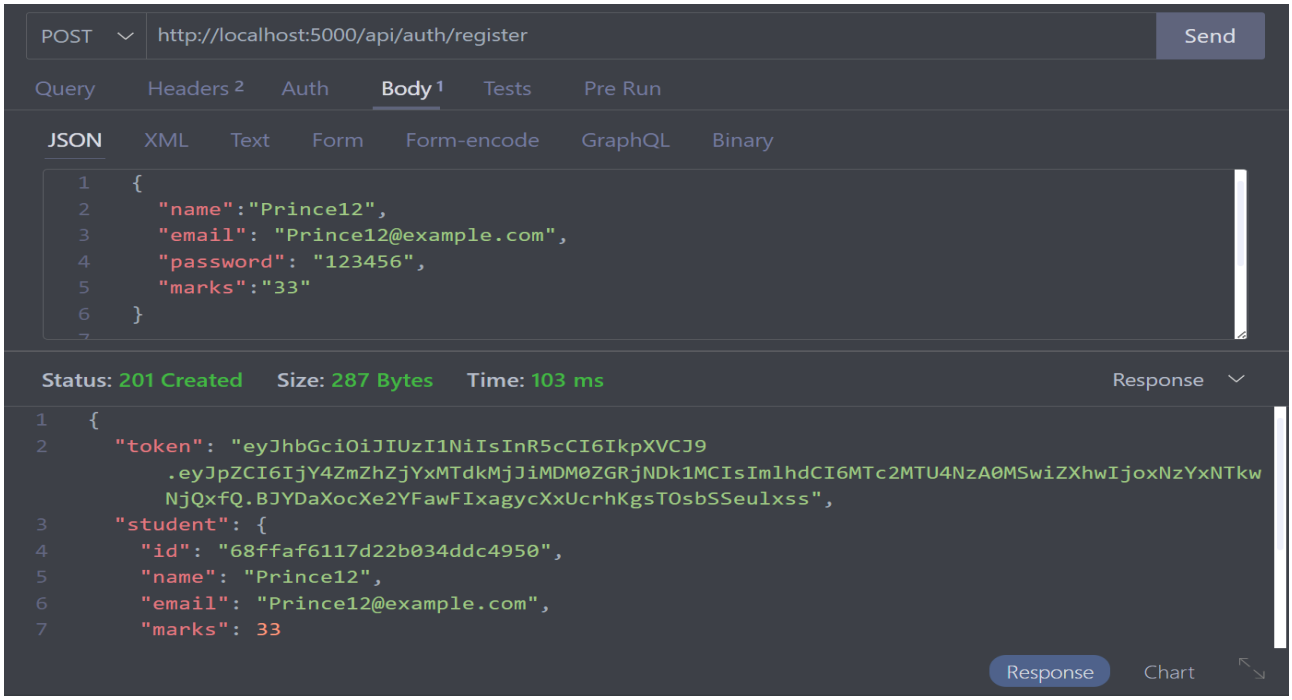
```
const jwt = require('jsonwebtoken');
const JWT_SECRET = process.env.JWT_SECRET || 'secretKey';
```

```
module.exports = function (req, res, next) {
  // Token can be in header 'x-auth-token' or Authorization Bearer token
  const token = req.header('x-auth-token') || (req.header('authorization') &&
req.header('authorization').split(' ')[1]);

  if (!token) return res.status(401).json({ msg: 'No token, access denied' });

  try {
    const decoded = jwt.verify(token, JWT_SECRET);
    req.user = decoded; // { id: ... , iat:..., exp:... }
    next();
  } catch (err) {
    return res.status(401).json({ msg: 'Token is not valid' });
  }
};
```

4. Result/Output:



5. Learning outcomes (What I have learnt):

- Connect Node.js applications with MongoDB using Mongoose.
- Apply password hashing for data security.
- Test and debug backend APIs effectively using Postman.